Mathematical Methods of Operations Research (1996) 43:1–8

Single Machine Batch Scheduling to Minimize the Weighted Number of Late Jobs[1]

PETER BRUCKER

Universität Osnabrück, 49069 Osnabrück, Germany

MIKHAIL Y. KOVALYOV

Institute of Engineering Cybernetics, Surganova 6, 220012 Minsk, Belarus

Abstract: The single machine batch scheduling problem to minimize the weighted number of late jobs is studied. In this problem, n jobs have to be processed on a single machine. Each job has a processing time, a due date and a weight. Jobs may be combined to form batches containing contiguously scheduled jobs. For each batch, a constant set-up time is needed before the first job of this batch is processed. The completion time of each job in the batch coincides with the completion time of the last job in this batch. A job is late if it is completed after its due date. A schedule specifies the sequence of jobs and the size of each batch, i.e. the number of jobs it contains. The objective is to find a schedule which minimizes the weighted number of late jobs. This problem is *NP*-hard even if all due dates are equal. For the general case, we present a dynamic programming algorithm which solves the problem with equal weights in $O(n^3)$ time. We formulate a certain scaled problem and show that our dynamic programming algorithm applied to this scaled problem provides a fully polynomial approximation scheme for the original problem. Each algorithm of this scheme has a time requirement of $O(n^3/\varepsilon + n^3 \log n)$. A side result is an $O(n \log n)$ algorithm for the problem of minimizing the maximum weight of late jobs.

Key Words: single machine batch scheduling, weighted number of late jobs, fully polynomial approximation scheme.

1 Introduction

The single machine batch scheduling problem to minimize the weighted number of late jobs (BSW problem) may be stated as follows. There are n jobs to be processed. Each job j becomes available for processing at time zero, requires a processing time $p_j > 0$, has a weight $w_j > 0$ and a due date $d_j \geq 0$. Jobs may be combined to form batches containing contiguously scheduled jobs. For each batch, a machine set-up time $s \geq 0$ is required before the first job of this batch is processed. We assume that all data are integer. The machine can handle only

[1] Supported by INTAS Project 93-257.

0340–9422/96/43:1/1–8 $2.50

one job at a time and cannot process any jobs while a set-up is being performed. A schedule specifies the sequence of jobs and the size of each batch, i.e. the number of jobs it contains. For any schedule, the completion time C_j of job j coincides with the completion time of the last job in its batch. If $C_j \leq d_j$ then job j is *early*; alternatively, if $C_j > d_j$, it is *late*. The objective is to find a schedule which minimizes the weighted number of late jobs $\sum_{j=1}^{n} w_j U_j$ where $U_j = 1$ if $C_j > d_j$ and $U_j = 0$ if $C_j \leq d_j$.

For a practical example, consider production of different parts on a multi-purpose machining centre. Parts are mounted on pallets for machining. Technology requirement is that parts mounted on the same pallet are processed together and will be available for shipment to customers only when all parts on the pallet have finished processing. A due date is associated with each part. The relative importance of each part is given by its weight. The objective is to minimize the total weighted number of parts not completed in time. This situation can be modelled as our single machine batch scheduling problem.

There has been recent interest in batch scheduling problems. For example, a single machine batch scheduling problem to minimize the total weighted job completion time was studied by Albers and Brucker [1], Brucker [2], Coffman et al. [6], [7], Dobson et al. [8], Naddef and Santos [14], while batch delivery single machine scheduling problems were analyzed by Cheng and Gordon [3], Cheng and Kahlbacher [5], Cheng et al. [4]. A review of this area of research was recently provided by Potts and Van Wassenhove [15].

Research of the BSW problem was initiated by Hochbaum and Landy [10]. They showed that the problem with nonnegative set-up time $s \geq 0$ and a common due date is NP-hard since the NP-hard problem of sequencing n jobs to minimize the tardy task weight (Karp [11]) is an instance of this problem for $s = 0$. We note that a simple transformation from the NP-complete problem PARTITION (Garey and Johnson [9]) shows that the BSW problem is also NP-hard if the set-up time is strictly positive. Hochbaum and Landy [10] presented a dynamic programming algorithm for the general problem and polynomial time algorithms for the cases when all processing times are equal or all weights are equal. In the current paper, we concentrate on developing an alternative dynamic programming algorithm and a fully polynomial approximation scheme.

We now comment on the structure of an optimal schedule. Without loss of generality, we assume that all late jobs are scheduled in an arbitrary order after the last early job. Furthermore, all early jobs can be processed in nondecreasing order of their due dates (see Hochbaum and Landy [10]). It is convenient to number jobs in EDD order so that $d_1 \leq \cdots \leq d_n$.

We call batches of early jobs *early batches*. Thus, the problem is reduced to finding a set of early jobs and defining sizes of early batches.

The rest of the paper is organized as follows. In the next section, we present a dynamic programming algorithm $DP(B)$ for the general problem where B is a guess for an upper bound on the minimum weighted number of late jobs W^*. Algorithm $DP(B)$ solves the problem if and only if $W^* \leq B$. It runs in $O(n^2B)$ time. The running time of the dynamic batching procedure presented by

Hochbaum and Landy [10] is $O(n^2 \min\{d_{\max}, (P + ns)\})$ where $d_{\max}$ is the value of the largest due date and P is the sum of processing times. Clearly, algorithm $DP(n)$ solves the problem with equal weights in $O(n^3)$ time. Hochbaum and Landy [10] presented an $O(n^4)$ algorithm for this problem.

Let L and V be valid lower and upper bounds for W^*: $L \leq W^* \leq V$. In Section 3, we formulate the problem with scaled weights $z_j = \lfloor nw_j/(\varepsilon L) \rfloor$ and show that algorithm $DP(nV/(\varepsilon L))$ applied to this scaled problem provides a $(1 + \varepsilon)$-approximation algorithm for the original problem. We denote this algorithm by $A_\varepsilon(L, V)$. The time complexity of $A_\varepsilon(L, V)$ is $O(n^3 V/(\varepsilon L))$. To find L and V such that $V = nL$, we present an $O(n \log n)$ algorithm for the problem of minimizing the maximum weight of late jobs. If W^0 is the optimal objective value of this problem then $W^0 \leq W^* \leq nW^0$. The running time of algorithm $A_\varepsilon(W^0, nW^0)$ is $O(n^4/\varepsilon)$. Bounds W^0 and nW^0 can be improved by using a Bound Improvement Procedure presented by Kovalyov [12]. This procedure runs in $O(n^3 \log n)$ time and delivers a value F^0 such that $F^0 \leq W^* \leq 3F^0$. Algorithm $A_\varepsilon(F^0, 3F^0)$ will run in $O(n^3/\varepsilon)$ time. The family of algorithms $\{A_\varepsilon(F^0, 3F^0)\}$ is our superior fully polynomial approximation scheme for the original problem. Some concluding remarks are presented in the last section.

2 Dynamic Programming

Let W^* be the optimal objective value for the BSW problem and let B be a positive integer. In this section, we present a dynamic programming algorithm $DP(B)$ which either solves the problem or establishes that $W^* > B$.

Assume the jobs are numbered in EDD order so that $d_1 \leq, \ldots, \leq d_n$. In algorithm $DP(B)$, jobs are considered in natural order $1, \ldots, n$. Three possible scheduling choices for each job are realized in the algorithm. They are the following:

- job j is scheduled as a late job,
- job j is scheduled at the end of the last early batch and will be completed by the earliest due date in that batch,
- job j is assigned to a new batch and will be completed before the due date d_j.

In $DP(B)$, the completion time of the last early job is a function value, while the weighted number of late jobs and the earliest due date in the last early batch are state variables. More precisely, we recursively compute values $C_j(W, d)$ which represent the minimum completion time of the last early job subject to j jobs are scheduled, the weighted number of late jobs is equal to W, and the earliest due date in the last early batch is equal to d. A formal statement of this dynamic programming algorithm is as follows.

Algorithm DP(B)

Step 1: (Initialization) Number jobs in EDD order. Set $C_j(W, d) = \infty$ for all $j = 0, 1, \ldots, n$, $W \in \{-w_{\max}, -w_{\max} + 1, \ldots, B\}$, where $w_{\max} = \max\{w_j | j = 1, \ldots, n\}$, and $d \in \{d_1, \ldots, d_n\}$. Set $C_0(0, d) = 0$ for all $d \in \{d_1, \ldots, d_n\}$. Set $j = 1$.

Step 2: (Recursion) Compute the following for all $W \in \{0, \ldots, B\}$ and $d \in \{d_1, \ldots, d_n\}$:

$$C_j(W, d) = \min \begin{cases} C_{j-1}(W - w_j, d) , & \\ C_{j-1}(W, d) + p_j , & \text{if } C_{j-1}(W, d) + p_j \leq d \\ & \text{and } C_{j-1}(W, d) > 0 , \\ K , & \text{if } d = d_j , \end{cases}$$

where $K = \min\{C_{j-1}(W, b) + s + p_j | C_{j-1}(W, b) + s + p_j \leq d_j, \ b \in \{d_1, \ldots, d_{j-1}\}\}$. If the latter minimum is taken over the empty set, then set $K = \infty$.

The three quantities in the right hand side of the recursion equation correspond to the three possible scheduling choices for each job j. If $j = n$, go to Step 3; otherwise set $j = j + 1$ and repeat Step 2.

Step 3: (Optimal solution) If $C_n(W, d) = \infty$ for all $W \in \{0, \ldots, B\}$ and $d \in \{d_1, \ldots, d_n\}$ then $W^* > B$; otherwise define

$$W^* = \min\{W | C_n(W, d) < \infty, W \in \{0, \ldots, B\}, d \in \{d_1, \ldots, d_n\}\}$$

and use backtracking to find corresponding optimal schedule.

It is apparent that algorithm $DP(B)$ runs in $O(n^2B)$ time since there are n different value of j, $B + 1$ different values of W and n different values of d. We now establish the correctness of this algorithm.

Theorem 1: Algorithm DP(B) solves the BSW problem if and only if $W^* \leq B$.

Proof: General dynamic programming justification (see Lawler and Moore [13], Rothkopf [16]) shows that at Step 2 of the algorithm, all possible objective values $W \leq B$ are constructed. At Step 3, if $C_n(W, d) = \infty$ for all $0 \leq W \leq B$ and $d \in \{d_1, \ldots, d_n\}$, then there are no solutions with value $W \leq B$, i.e. $W^* > B$ that proves necessity. Alternatively, if $C_n(W, d) < \infty$ for certain W and d, then W^* is the minimal objective value among those obtained at Step 2. Thus, the sufficiency is also proved. ■

Clearly, $DP(W_\Sigma)$, where W_Σ is the sum of all weights, solves the BSW problem in $O(n^2 W_\Sigma)$ time. Thus, the problem with equal weights is solved by $DP(n)$ in $O(n^3)$ time which is better than the $O(n^4)$ algorithm presented by Hochbaum and Landy [10].

3 A Fully Polynomial Approximation Scheme

In this section, we present a fully polynomial approximation scheme for the BSW problem. Recall, that an algorithm A_ε for this problem is a $(1+\varepsilon)$-*approximation algorithm* if we have $W \leq (1+\varepsilon)W^*$ for all instances, where W^* denotes the optimal solution value and W denotes the value of the solution given by the algorithm. A family of approximation algorithms $\{A_\varepsilon\}$ defines a *fully polynomial approximation scheme* if, for any $\varepsilon > 0$, A_ε is a $(1+\varepsilon)$-approximation algorithm which is polynomial in n and $1/\varepsilon$.

To construct A_ε, let L and V be numbers satisfying $L \leq W^* \leq V$. Furthermore, we set $\delta = \varepsilon L/n$ and define scaled weights $z_j = \lfloor w_j/\delta \rfloor$ for $j = 1, \ldots, n$. Firstly, we will define a procedure $A_\varepsilon(L, V)$ from which A_ε is derived by choosing appropriate values L and V.

$A_\varepsilon(L, V)$ may be described as follows. We apply $DP(V/\delta)$ to the BSW problem with scaled weights, which provides an optimal solution value $V' := \sum_{j \in I} z_j$, where I is the set of late jobs. $A_\varepsilon(L, V)$ chooses I as a solution for the original problem, i.e. the solution value of $A_\varepsilon(L, V)$ is $W' := \sum_{j \in I} w_j$. Making use of the inequalities $\lfloor x \rfloor \leq x < \lfloor x \rfloor + 1$, we have

$$V' \leq W^*/\delta \leq V/\delta$$

and

$$W' \leq \delta V' + n\delta \leq W^* + n\delta \leq (1+\varepsilon)W^* \ .$$

The latter inequalities show that $A_\varepsilon(L, V)$ is a $(1+\varepsilon)$-approximation algorithm for the original problem and the former inequalities show that V/δ is a valid upper bound for the optimal value V' of the scaled problem. This implies that the running time of $A_\varepsilon(L, V)$ is $O(n^3 V/(\varepsilon L))$.

To find appropriate values for L and V, we consider the problem of minimizing the maximum weight of late jobs $\max\{w_j | j \text{ is late}\}$. Let W^0 be the minimal solution value of this problem and let I^0 be the set of late jobs in the corresponding schedule. If I^0 is empty, then this schedule is optimal for the original BSW

problem and we have $W^* = 0$. Assume I^0 is not empty and, consequently, $W^0 > 0$ and $W^* > 0$. Let I^* be the set of late jobs in an optimal schedule for the BSW problem. Then the following chain of relations holds:

$$W^0 \leq \max\{w_j | j \in I^*\} \leq \sum_{j \in I^*} w_j = W^* \leq \sum_{j \in I^0} w_j \leq nW^0 \ .$$

Thus, we have $W^0 \leq W^* \leq nW^0$. Define $L = W^0$ and $V = nW^0$. Then algorithm $A_\varepsilon := A_\varepsilon(W^0, nW^0)$ will run in $O(n^4/\varepsilon)$ time. Hence, the family of algorithms $\{A_\varepsilon\}$ forms a fully polynomial approximation scheme for the BSW problem.

We note that algorithm $A_\varepsilon(L, V)$ has properties which allow us to apply the Bound Improvement Procedure presented by Kovalyov [12] for finding a value F^0 such that $F^0 \leq W^* \leq 3F^0$. In iteration $k = 1, 2, \ldots$ of this procedure, algorithm $A_1(F, 2F)$ is applied to the BSW problem, where $F = 2^{k-1}W^0$. If it finds a solution, then $F \leq W^* \leq 3F$ and the procedure is terminated. Otherwise, $W^* > 2F$ is satisfied and the procedure continues for $k = k + 1$. Since $W^0 \leq W^* \leq nW^0$, the number of iterations of this procedure does not exceed $O(\log n)$ time. The running time of $A_1(F, 2F)$ is $O(n^3)$. Therefore, the procedure runs in $O(n^3 \log n)$ time. Algorithm $A_\varepsilon(F^0, 3F^0)$ will run in $O(n^3/\varepsilon)$ time. Thus, the overall time complexity of each algorithm in our superior fully polynomial approximation scheme is $O(n^3/\varepsilon + n^3 \log n)$.

It remains to show how to calculate W^0. Let $(i_1, \ldots, i_n)$ be a sequence of jobs given in nonincreasing order of their weights: $w_{i_1} \geq w_{i_2} \geq \cdots \geq w_{i_n}$. It is apparent that $W^0 = w_{i_{(m+1)}}$ where m is the maximum index k for which all jobs $i_1, \ldots, i_k$ can be early in some schedule. If $m = n$ then $W^0 = 0$.

To find the index m, we can perform a binary search in the range $1, \ldots, n$ where for each trial value k the question "can all jobs $i_1, \ldots, i_k$ be early?" has to be answered. It is shown by Hochbaum and Landy [10] that this question can be answered in $O(n)$ time as follows.

Renumber the jobs so that $d_{i_1} \leq \cdots \leq d_{i_k}$. If we sort the jobs according to nondecreasing d_j-values in a preprocessing step then this renumbering can be done in $O(n)$ time. Assign jobs in order $1, \ldots, k$ to the end of the current schedule. Let $j - 1$ jobs be assigned. If jobs $1, \ldots, j$ can be scheduled early by adding j to the last batch, then do so. If not, but jobs $1, \ldots, j$ can be early by starting a new batch containing only job j, then do so. If neither, then there is no schedule in which jobs $1, \ldots, k$ can be early. By maintaining the value of the completion time of the last job and the value of the earliest due date in the last batch, each iteration of this recursive procedure can be performed in constant time. Since the number of iterations of this procedure is at most k, the question can be answered in $O(n)$ time. Thus, we establish the following result.

Theorem 2: The problem of minimizung the maximum weight of late jobs can be solved in $O(n \log n)$ time.

Proof: Sorting n jobs in nonincreasing order of their weights can be done in $O(n \log n)$ time. The binary search for the value of m requires $O(\log n)$ steps, where each step can be done in $O(n)$ time. Thus, the overall time complexity is $O(n \log n)$.

4 Conclusion

We investigated the single machine batch scheduling problem with weighted number of late jobs as objective function. The study of this problem was initiated by Hochbaum and Landy [10]. They proved the *NP*-hardness of this problem when set-up time $s \geq 0$ by showing that the problem with $s = 0$ is *NP*-hard. They presented a dynamic programming algorithm for the general problem and polynomial time algorithms for the cases when all processing times are equal or all weights are equal. We presented an alternative dynamic programming algorithm. When all weigths are equal, our algorithm solves the problem in $O(n^3)$ time which is better than the $O(n^4)$ algorithm presented by Hochbaum and Landy [10].

The main result of this paper is a fully polynomial approximation scheme for the original problem. Each algorithm of this scheme has a time requirement of $O(n^3/\varepsilon + n^3 \log n)$. A side result is an $O(n \log n)$ algorithm for the problem of minimizing the maximum weight of late jobs.

References

[1] Albers S, Brucker P (1993) The complexity of one-machine batching problems. Discrete Applied Mathematics 47:87–107

[2] Brucker P (1991) Scheduling problems in connection with flexible production systems. In Proceedings, 1991 IEEE Int Conf on Robotics and Automation 1778–1783

[3] Cheng TCE, Gordon VS (1992) On batch delivery scheduling on a single machine, Preprint N 9, Institute of Engineering Cybernetics, Belarus Academy of Sciences, Minsk

[4] Cheng TCE, Gordon VS, Kovalyov MY (1994) Single machine scheduling with batch deliveries. (in submission)

[5] Cheng TCE, Kahlbacher HG (1993) Scheduling with delivery and earliness penalties. Asia-Pacific Journal of Operational Research 10:145–152

[6] Coffman EG, Nozari A, Yannakakis M (1989) Optimal scheduling of products with two subassemblies on a single machine. Operations Research 37:426–436

[7] Coffman EG, Yannakakis M, Magazine MJ, Santos C (1990) Batch sizing and sequencing on a single machine. Annals of Operations Research 26:135–147

[8] Dobson G, Karmarkar US, Rummel JL (1987) Batching to minimize flow times on one machine. Management Science 33:784–799

[9] Garey MR, Johnson DS (1979) Computers and intractability: A guide to the theory of *NP*-completenss. W.H. Freeman and Co, New York
[10] Hochbaum DS, Landy D (1994) Scheduling with batching: minimizing the weighted number of tardy jobs. (to appear in Operations Research Letters)
[11] Karp RM (1972) Reducibility among combinatorial problems. In: Miller RE, Thatcher JW (eds) Complexity of Computer Computations. Plenum Press, New York 85–103
[12] Kovalyov MY (1994) Improving the complexities of approximation algorithms for optimization problems. Operations Research Letters 16:79–86
[13] Lawler EL, Moore JM (1969) A functional equation and its application to resource allocation and sequencing problems. Management Science 16:77–84
[14] Naddef D, Santos C (1988) One-pass batching algorithms for the one machine problem. Discrete Applied Mathematics 21:133–145
[15] Potts CN, Van Wassenhove LN (1991) Integrating scheduling with batching and lot-sizing: A review of algorithms and complexity. Journal of the Operational Research Society 43:395–406
[16] Rothkopf MH (1966) Scheduling independent tasks on parallel processors. Management Science 12:437–447

Received: December 1993
Revised version received: January 1995